

N706B OpenCPU

SDK Application Guide

Issue 1.1
Date 2025-05-14



Copyright © Neoway Technology Co., Ltd. 2025. All rights reserved.

No part or whole of this document may be reproduced or transmitted by any individual or other entity in any form or by any means without prior written consent of Neoway Technology Co., Ltd.

neoway is the trademark of Neoway Technology Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

This document is specifically for N706B OpenCPU.

This document is intended for system engineers (SEs), development engineers, and test engineers.

THIS DOCUMENT PROVIDES SUPPORT FOR PRODUCT DESIGN BY THE USER. THE USER MUST DESIGN AND DEBUG THE PRODUCT ACCORDING TO THE SPECIFICATIONS AND PARAMETERS IN THIS DOCUMENT. NEOWAY ASSUMES NO RESPONSIBILITY FOR PERSONAL INJURY AND PROPERTY LOSS CAUSED BY USER'S IMPROPER OPERATION.

DUE TO PRODUCT VERSION UPDATES OR OTHER REASONS, THE CONTENT OF THIS DOCUMENT WILL BE UPDATED AS NECESSARY WITHOUT PRIOR NOTICE.

UNLESS OTHERWISE AGREED, ALL STATEMENTS, INFORMATION AND SUGGESTIONS IN THIS DOCUMENT CONSTITUTE NO WARRANTY WHETHER EXPRESS OR IMPLIED.

Neoway Technology Co., Ltd. provides customers with comprehensive technical support. For any inquiry, please contact your account manager directly or send an email to the following email address:

Sales@neoway.com

Support@neoway.com

Website: <http://www.neoway.com>

Contents

1 Overview	5
1 Resource Introduction	6
1.2 System Introduction	6
1.2.1 ThreadX Task Fundamentals	6
1.2.2 Multitasking System	6
1.3 Interface Introduction	8
1.4 Common Tools and Documentation Description	9
2 Setting Up Development Environment	11
2.1 Compilation Environment	11
2.2 Development Environment	11
2.3 Debug Environment	13
2.4 Environmental Verification	14
2.4.1 Windows Script Compilation	14
2.4.2 Compilation Log Output	15
2.4.3 File Burning	16
2.4.4 Application Debugging	17
3 Introduction to SDK Directory	19
4 Application Example	21
4.1 Code Entry	21
4.2 Task Creation	22
4.3 Serial Port Application	22
4.3.1 Serial Port demo Instructions	22
4.3.2 Serial Port Test Example	24
4.4 Timer	26
4.5 FLASH Address Related	27
5 Debugging	28

About This Document

Scope

This document is applicable to N706B OpenCPU series.

Audience

This document is intended for [system engineers \(SEs\)](#), [development engineers](#), and [test engineers](#).

Change History

Issue	Date	Change	Changed By
1.0	2025-01	Initial release	Liam
1.1	2025-05	Updated compilation script	Clara Xue

Conventions

Symbol	Indication
	This warning symbol means danger. You are in a situation that could cause fatal device damage or even bodily damage.
	Means reader be careful. In this situation, you might perform an action that could result in module or product damages.
	Means note or tips for readers to use the module

1 Overview

The N706B (ASR1605 Cat.1) open platform module operates on the ThreadX operating system. This document primarily introduces methods for setting up the N706B OpenCPU development environment, code development, and APP debugging. It can be used to guide the development and application of the N706B OpenCPU module.

Requirements for developers:

- Understand the principles of the ThreadX system.
- Master the standard C language application development methods.
- Master ThreadX thread synchronization and related communication knowledge.
- Familiar with standard protocols such as TCP, FTP, HTTP, UART, SPI, I2C, and the usage methods of module peripherals.

1 Resource Introduction

The basic functions supported by N706B OpenCPU are as follows:

Table 1-1 Features supported by N706B OpenCPU

Category	Description
System	ThreadX
CPU	ARM Cortex-R5 processor, 614MHz main frequency.
Modem	FDD-LTE: Cat 1, Max 10 Mbps (DL)/Max 5 Mbps (UL) TDD-LTE: Cat 1, Max 8 Mbps (DL)/Max 2 Mbps (UL)
OS	Threadx
Memory	N706B-XX-1X model integrates 32Mb FLASH and 32Mb RAM N706B-XX-2X model integrates 16Mb FLASH and 32Mb RAM
Peripheral	Supports USB2.0 / USIM / ADC / UART / GPIO / SPI / I2C / GNSS (optional)
Protocol stack	TCP, UDP, MQTT, FTP, HTTP/HTTPS, SSL, TLS
Location	Model N706B-CN-10 supports GNSS

1.2 System Introduction

ThreadX is a lightweight real-time operating system with features including task management, time management, semaphores, message queues, memory management, logging functions, software timers, coroutines, and more, which can basically meet the needs of smaller systems.

1.2.1 ThreadX Task Fundamentals

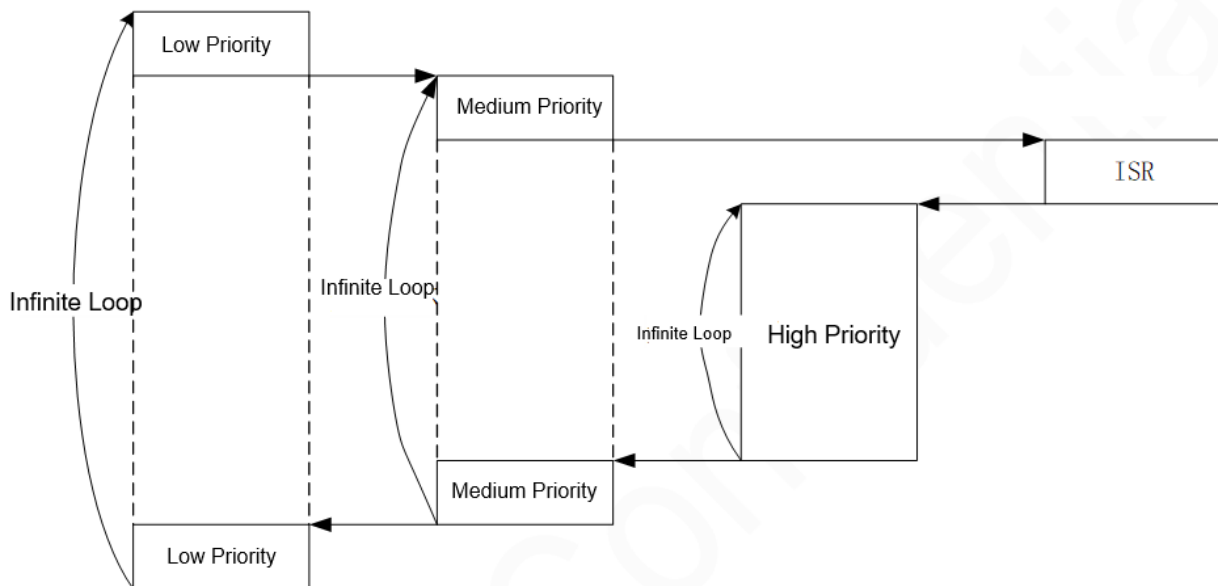
The core of the ThreadX system is task management. When getting started with an RTOS system, the first thing you must master is the creation, deletion, suspension, and resumption of tasks, highlighting the importance of task management.

1.2.2 Multitasking System

A multitasking system can break a big problem (application) down into many smaller problems. By gradually solving these smaller problems, the big problem is also resolved. These smaller problems

can be handled individually as separate tasks and they are processed concurrently. Note that this does not mean executing many tasks simultaneously at the same moment, but rather, because each task's execution time is very short, it appears as if many tasks are being executed at the same moment. Multiple tasks have introduced a new problem: which task should run first and which should run later. The component that accomplishes this function in an RTOS system is called a task scheduler. The implementation methods of task schedulers vary across different systems. For example, ThreadX is a preemptive real-time multitasking system, so its task scheduler is also preemptive. The operation process is shown in Figure 2-2:

Figure 1-2 Running process



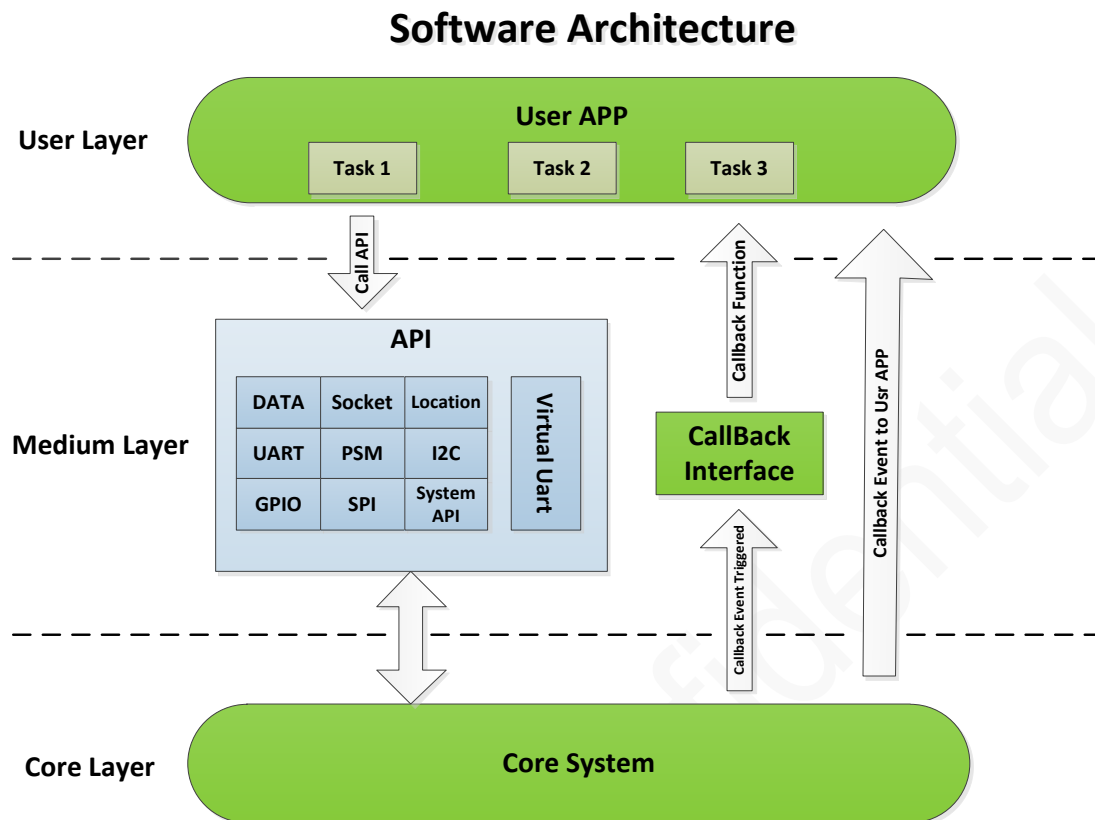
Notes:

When no high-priority tasks exist, execute tasks within the current priority level.

When a higher-priority task appears, the current task is suspended and waits for the higher-priority task to complete.

In the diagram, high-priority tasks can interrupt the execution of low-priority tasks to gain control of the CPU, thereby ensuring the execution of urgent tasks. This way, we can set a very high priority for tasks that require real-time processing, such as obstacle detection in autonomous driving. After the high-priority task is completed, the CPU usage is returned to the low-priority task. This is the basic principle of a preemptive multitasking system.

Figure 1-3 Introduction to the open platform



Neoway OpenCPU platform architecture is shown in Figure 1-3. The platform is mainly divided into three layers:

- System Driver Layer**
 Involving related drivers at the lower level, primarily for the application programming interface calls and interactions of the intermediate interface layer. At the same time, for user convenience, a few lower-level interfaces are directly open for user use.
- Intermediate Interface Layer**
 Mainly includes: GPIO configuration interface, interrupt interface, I2C interface, SPI interface, file system operation interface, timer interface, low-level callback interface, USB interface, and system interface, etc.
- User Application Layer**
 Mainly used for customer application development to implement business logic control.

1.3 Interface Introduction

This section mainly introduces the hardware resources and software interfaces of the N706B module.

For detailed interface descriptions, please refer to the N706B Hardware User Guide, Pin Multiplexing Table, API Interface Description, and other documents.

API Classification	Function Introduction
Peripheral	Enabling and invoking the I/O port, including • UART (The N706B module provides 3 serial ports: two serial ports for peripheral data transmission, and one debug serial port for capturing module AP logs) • I2C (Master mode, two channels available) • SPI (Master mode, One channel available) • GPIO (approximately 10, depending on multiplexing) • ADC (two channels available)
Services	Enabling and invoking network service interfaces, including: • Data (Data dial) • SIM (SIM card service) • SMS (Short message service) • Location (Positioning service) • Wi-Fi Scan (Wi-Fi functionality, only supports STA, primarily used for assisting indoor positioning) • Network (Network registration) • FOTA (Differential upgrade) • AT (Supports partial 3GPP standard commands) • Socket (Control socket)
Protocol interface	Common protocol interfaces: TCP, UDP, MQTT, FTP, HTTP/HTTPS, SSL, TLS The interface uses standard interfaces. For more details, you can search online or refer to the demo provided by Neoway.
Other interfaces	ThreadX message queue and timers, etc.

1.4 Common Tools and Documentation Description

NO	Name	Note
1	Neoway_N706B_EVK_User_Guide	Provides N706B EVB introduction and function description
2	Neoway_N706B_Log_Capture_Guide	Methods for capturing N706B module underlying logs
3	Neoway_N706B_USB_Driver_Installation_Guide	Introduces the method of installing USB drivers.

4	Neoway_N706B_Firmware_Upgrade_Guide	Introduce the method of firmware upgrade.
5	Neoway_N706B_and_N706-CB_Compatibility_Design_Guide	Provides N706B and N706-CB hardware compatibility design reference
6	PL2303_Prolific_DriverInstaller_v110.exe	Serial port driver
7	CATStudio	Log capturing tool
8	Aboot-tool	Upgrade and burning tool

2 Setting Up Development Environment

2.1 Compilation Environment

- System requirements

Windows 7, x64 or Windows 10, x64

- Cross-compilation tools

The tool files can be found in the SDK package under the /tool/ directory (included in the SDK, customers do not need to worry about the compilation environment, just download the SDK to a PC and extract it, then use the command line to compile).



The SDK storage path should only contain letters, numbers, or underscores; otherwise, it will cause compilation errors.

2.2 Development Environment

- System Requirements

Windows 7/8/10

- Programming Tools

Source insight (can be adjusted according to actual situation)

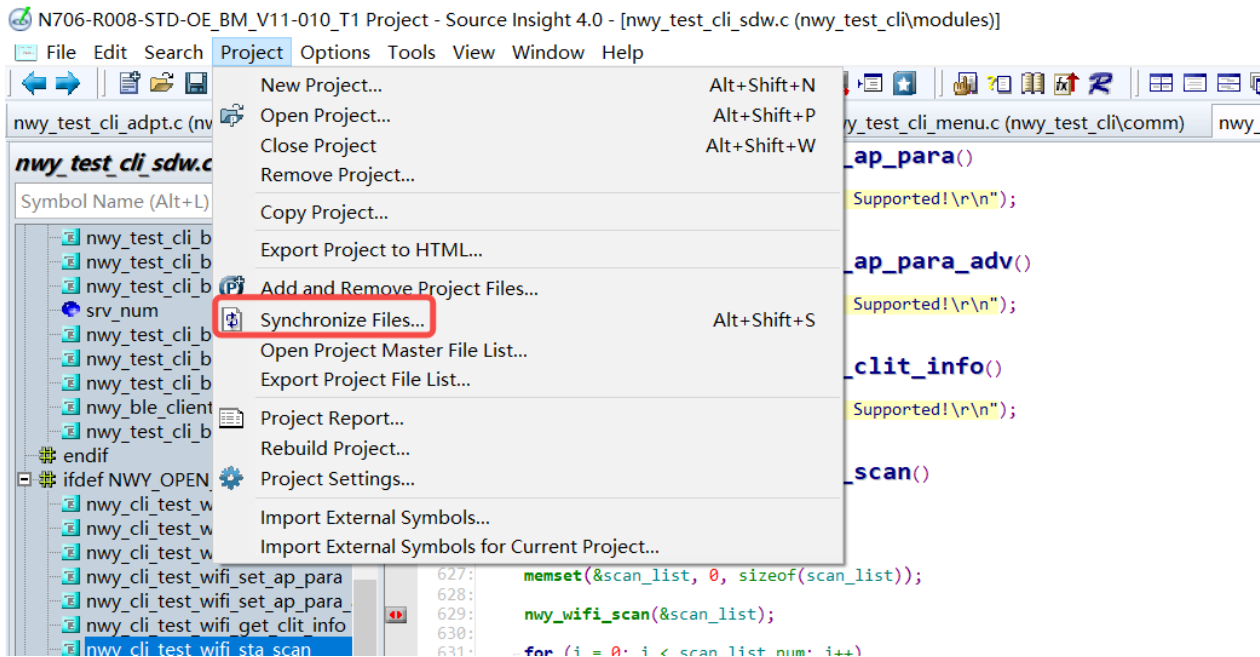


Since currently only compilation under Windows OS is supported, it is recommended to use Source Insight for programming. If the software is not available, you can consult Neoway FAE for assistance.

Source Insight Usage Process

1. After installing the corresponding software, create the corresponding project. You can find tutorials on the installation and use of Source Insight online.
2. Import the corresponding project files.

- To synchronize the program, click on 'Synchronize Files...' under the Project menu. This will allow you to find the interrelationships between the files within the project, making it easier to search and index.



There is a useful tip for applications: since the imported project files are numerous and include most of the compiler-related and other related files, we won't be using them in our daily debugging. Therefore, you can sort the files by directory in the navigation bar, making it easier to find the necessary files during the code development process. The root directory and the components directory are mostly related to the project.

Project Files %			
Project Symbols			
Folders			
File Name (Ctrl+O)			
File Name	Directory	Size	
nwy_test_cli_adpt.c	nwy_test_cli\cc	5273	^
nwy_test_cli_adpt.h	nwy_test_cli\cc	2930	
nwy_test_cli_driver.c	nwy_test_cli\m	58255	
nwy_test_cli_func_def.h	nwy_test_cli\m	12381	
nwy_test_cli_iot.c	nwy_test_cli\m	46712	
nwy_test_cli_lbs.c	nwy_test_cli\m	10346	
nwy_test_cli_menu.c	nwy_test_cli\cc	27532	
nwy_test_cli_mobile.c	nwy_test_cli\m	66419	
nwy_test_cli_os.c	nwy_test_cli\m	12954	
nwy_test_cli_sdw.c	nwy_test_cli\m	20419	
nwy_test_cli_tcpip.c	nwy_test_cli\m	47434	
nwy_test_cli_utils.c	nwy_test_cli\cc	5115	
nwy_test_cli_utils.h	nwy_test_cli\cc	917	
nwy_uart.h	components\ir	4759	
nwy_usb_serial.h	components\ir	1508	
nwy_vir_at.h	components\ir	2097	
nwy_wifi.h	components\ir	687	
N_005fBROWS.html	prebuilts\win3;	3494	
N_005fCATCH.html	prebuilts\win3;	3706	

2.3 Debug Environment

- System Requirement

Win 7/8/10

- Debugging Tools

The N706B OpenCPU debug log is primarily output from the Neoway USB OPEN_CON port and can be debugged using a standard serial port debugging tool. In this document, the SSCOM serial port tool is used for debugging.

- Installing USB Drivers

To install the USB driver on Windows, after the driver is successfully installed, 7 COM ports will appear. Their main functions are as follows:

No.	Port Name	Note
1	Neoway USB AP_TRACE	/
2	Neoway USB AT	AT port
3	Neoway USB CP_TRACE	/
4	Neoway USB Diagnostic	Module AP and CP log capture
5	Neoway USB Modem	Modem port, can be used for PPP dialing
6	Neoway USB NMEA	GPS Data data output port
7	Neoway USB OPEN_CON	N706B OpenCPU debug log output port





For detailed installation steps, please refer to the 'Neoway_N706B_USB_Driver_Installation_Guide'.

Please refer to the 'Neoway_N706B_Log_Capture_Guide' for capturing the N706B module AP log and CP log.

2.4 Environmental Verification

2.4.1 Windows Script Compilation

Step 1: Extract the SDK package.

build	5/7/2025 10:27 AM	File folder	
cmake	5/7/2025 10:27 AM	File folder	
doc	5/7/2025 10:27 AM	File folder	
doc_en	5/7/2025 10:27 AM	File folder	
include	5/7/2025 10:27 AM	File folder	
N706B.si4project	5/7/2025 4:53 PM	File folder	
nwy_test_demo	5/7/2025 10:27 AM	File folder	
out	5/8/2025 4:30 PM	File folder	
prebuilts	5/7/2025 10:27 AM	File folder	
tools	5/7/2025 10:28 AM	File folder	
build_open_app_cmake.bat	5/7/2025 10:27 AM	Windows Batch File	3 KB
CMakeLists.txt	5/7/2025 10:27 AM	Text Source File	6 KB
cmd	5/7/2025 10:27 AM	Shortcut	2 KB
readme.txt	5/7/2025 10:27 AM	Text Source File	2 KB

Step 2: Compile the app.

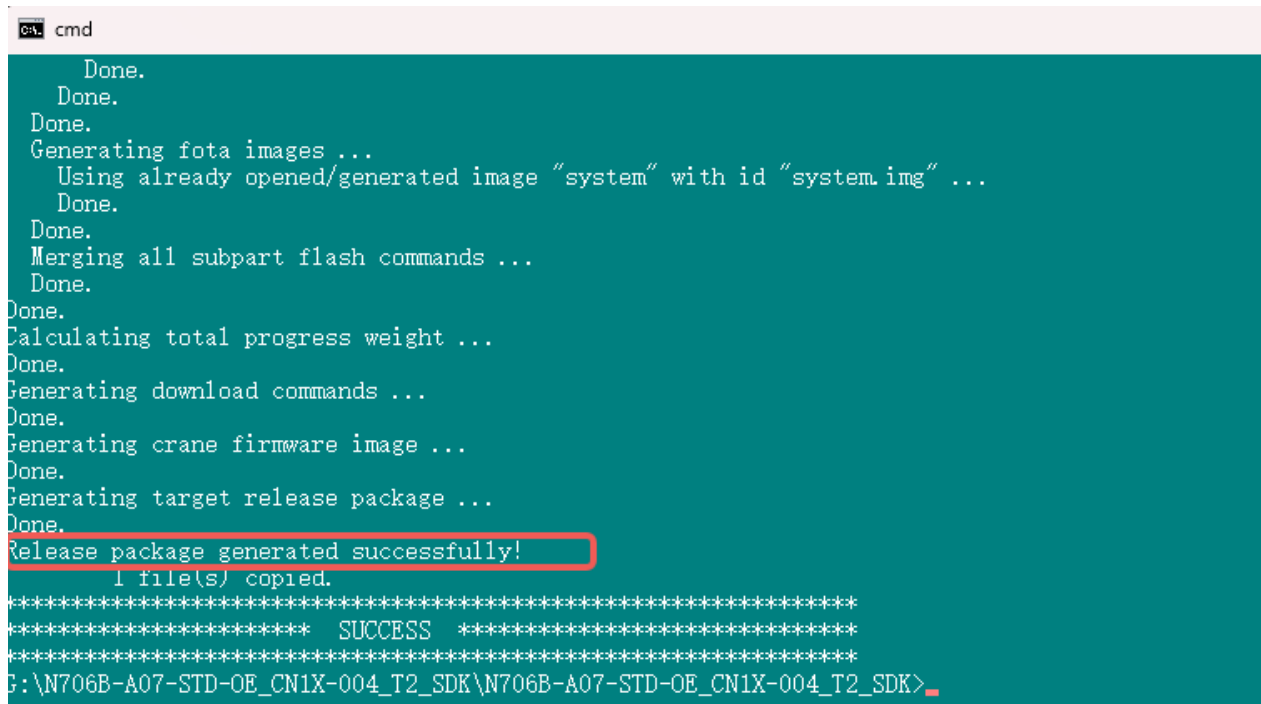
1. Double-click the cmd shortcut in the sdk directory.

名称	修改日期	类型	大小
build	2024/12/23 20:52	文件夹	
include	2024/12/23 20:50	文件夹	
nwy_hello_world	2024/12/23 20:50	文件夹	
nwy_test_demo	2024/12/23 20:50	文件夹	
tools	2024/12/23 20:50	文件夹	
build_open_app.bat	2025/1/8 14:21	Windows 批处理文件	2 KB
cmd	2024/12/12 21:09	快捷方式	2 KB
readme.txt	2025/1/8 14:18	文本文档	2 KB

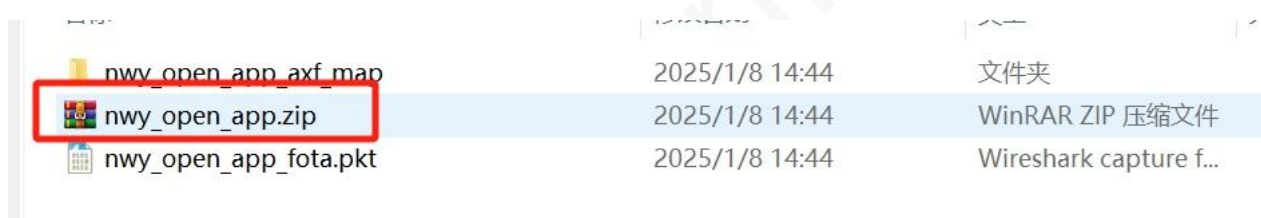
2. Execute the build_open_app_cmake.bat compilation script in the command line interface to start compiling.

```
cmd
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.
G:\N706B-A07-STD-OE_CN1X-004_T2_SDK\N706B-A07-STD-OE_CN1X-004_T2_SDK>build_open_app_cmake.bat
```

- After the compilation is complete, a 'successfully compiled' message will be displayed. You can find the generated nwy_open_app.zip file in the SDK's \release directory.



```
cmd
Done.
Done.
Done.
Generating fota images ...
Using already opened/generated image "system" with id "system.img" ...
Done.
Done.
Merging all subpart flash commands ...
Done.
Done.
Calculating total progress weight ...
Done.
Generating download commands ...
Done.
Generating crane firmware image ...
Done.
Generating target release package ...
Done.
Release package generated successfully!
1 file(s) copied.
*****
***** SUCCESS *****
*****
G:\N706B-A07-STD-OE_CN1X-004_T2_SDK\N706B-A07-STD-OE_CN1X-004_T2_SDK>
```

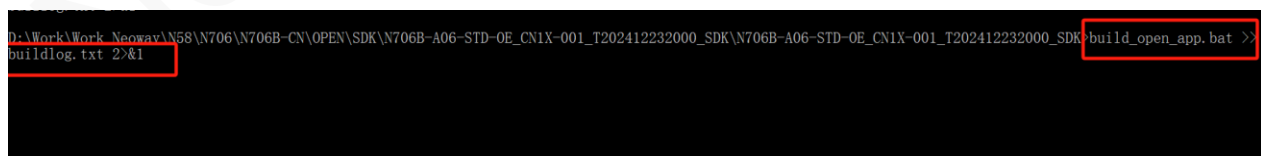


File Name	Time	Type
nwy_open_app_axf_map	2025/1/8 14:44	文件夹
nwy_open_app.zip	2025/1/8 14:44	WinRAR ZIP 压缩文件
nwy_open_app_fota.pkt	2025/1/8 14:44	Wireshark capture f...

2.4.2 Compilation Log Output

When you are doing secondary development, if there are too many errors in the written code, you can add a command to output compilation logs during the compilation process.

- In the aforementioned step 2, open the cmd command line and enter the command `build_open_app.bat >> buildlog.txt 2>&1`



```
D:\Work\Work_Neoway\N58\N706\N706B-CN\OPEN\SDK\N706B-A06-STD-OE_CN1X-001_T202412232000_SDK\N706B-A06-STD-OE_CN1X-001_T202412232000_SDK>build_open_app.bat >> buildlog.txt 2>&1
```

- Afterwards, the cmd screen will not output the compilation log information; all logs will be printed in the SDK directory.

build	2024/12/23 20:52	文件夹	
include	2024/12/23 20:50	文件夹	
nwy_hello_world	2024/12/23 20:50	文件夹	
nwy_test_demo	2024/12/23 20:50	文件夹	
release	2025/1/8 18:08	文件夹	
tools	2024/12/23 20:50	文件夹	
build_open_app.bat	2025/1/8 14:46	Windows 批处理文件	2 KB
buildlog.txt	2025/1/8 18:08	文本文档	71 KB
cmd	2024/12/12 21:09	快捷方式	2 KB
readme.txt	2025/1/8 14:18	文本文档	2 KB

2.4.3 File Burning



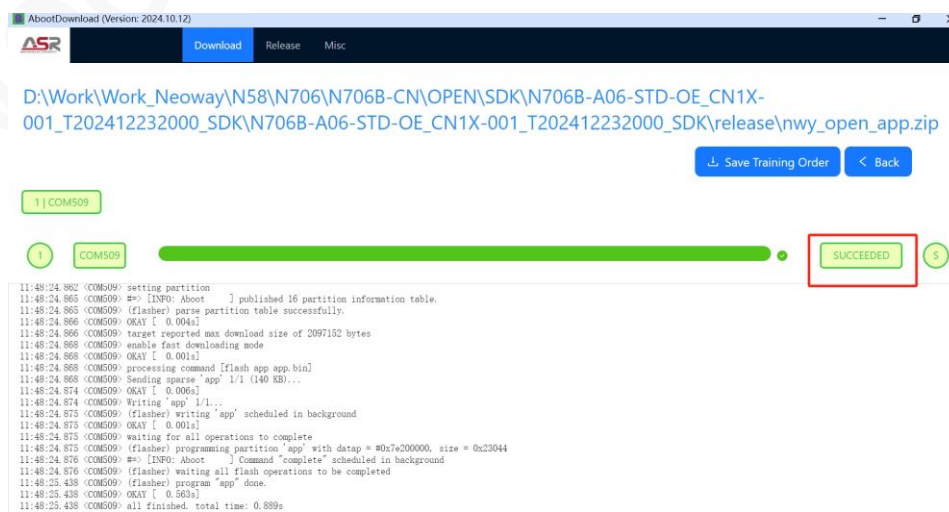
Before programming the APP, ensure that the N706B module already contains the N706B open firmware (you can use the AT+GMR command to check the firmware version and SDK consistency at the USB AT port). Otherwise, the secondary development program will fail to run, and the OPEN menu log cannot be printed. The underlying firmware is included in the SDK compressed package. For example, the file N706B-A06-STD-OE_CN1X-001.zip is an underlying firmware package. The flashing steps are the same as for the APP. During the development process, the firmware only needs to be flashed once.

- Size of the underlying firmware: approximately 4MB
- APP size: approximately 2MB

The APP generated from the secondary development of the N706B module needs to be flashed into the N706B module using the about-tools tool.

For specific programming steps, please refer to the 'Neoway_N706B_Firmware_Upgrade_Guide'.

If the main interface displays 'SUCCEEDED', it indicates that the APP has successfully imported the module. Close the burning tool, the module will automatically restart, and the APP will automatically run.



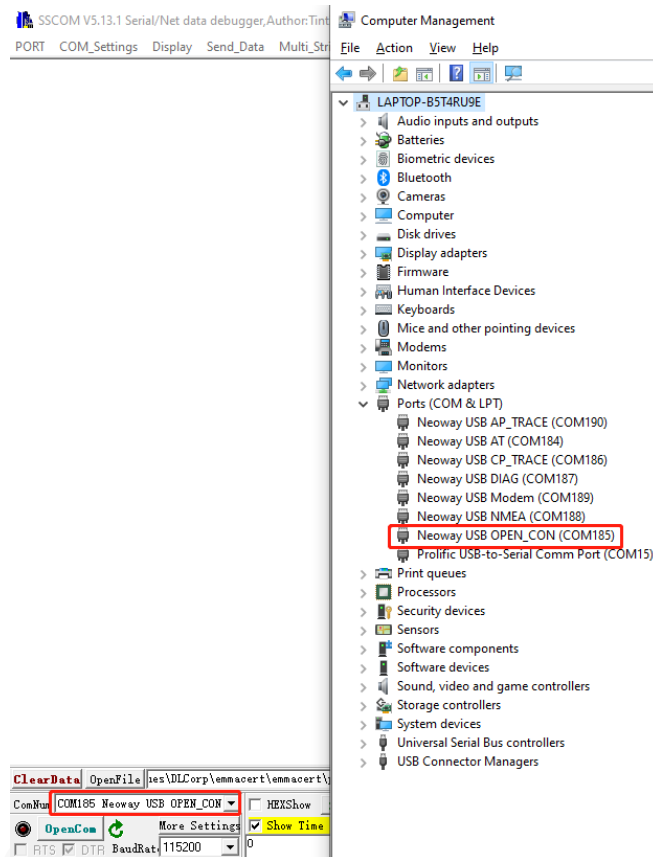
2.4.4 Application Debugging

The log output by the `nwy_ext_echo()` function will be printed through the USB OPEN port (Neoway USB_OPEN_CON).

Step 1: Open the SSCOM tool.

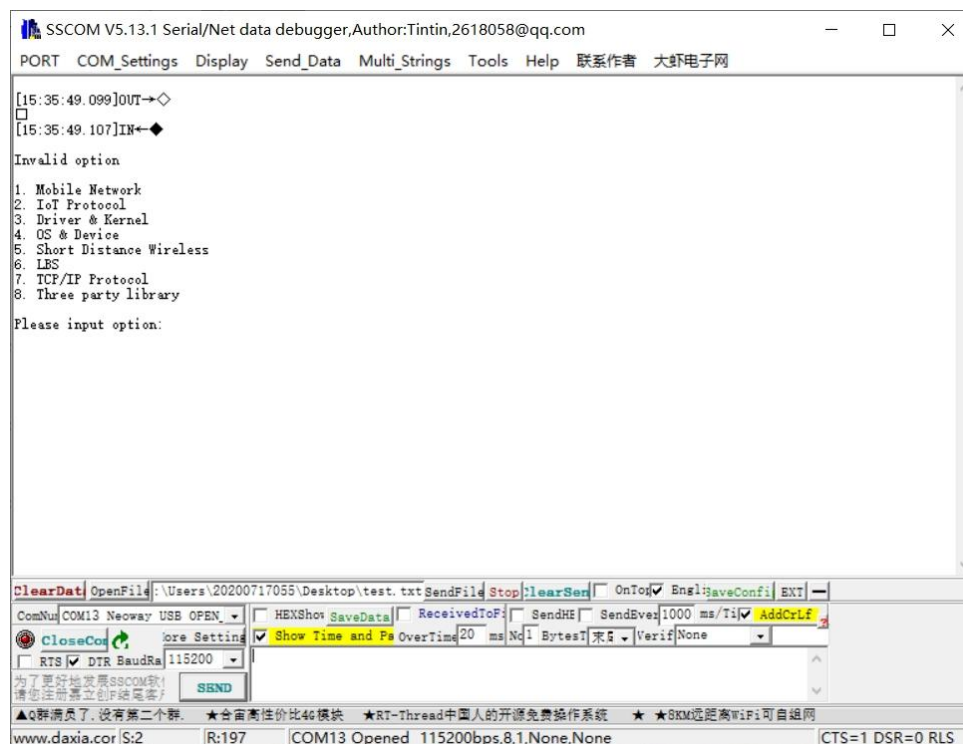
Step 2: Configure the relevant settings for this tool.

1. Set the connection type of the tool to 'Serial'.
2. Enter the USB port number in the Serial line input box.



Step 3: Send a space or any character in the SSCOM tool to verify if the Neoway_USB_OPEN_CON port is connected.

When the serial port tool receives the relevant log information, you can start debugging the APP.

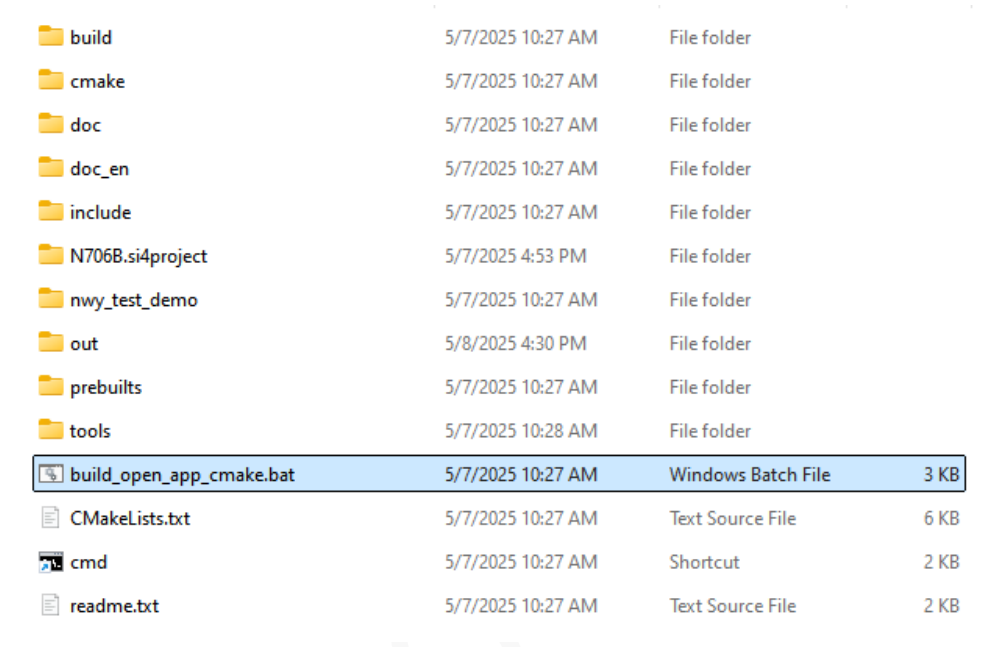


If the relevant log information is not received, you can troubleshoot the issue by following these two points:

- Use the AT+GMR command to check if the module firmware version matches the firmware version in the SDK. If the versions do not match, reflash the firmware.
- If it is confirmed that the firmware version corresponds to the APP, then refer to steps 2 of section 3.4.1 and section 3.4.2 to sequentially check whether the APP compilation and APP programming are successful.

3 Introduction to SDK Directory

After obtaining the SDK compressed package and extracting it, you will see the following:



build	5/7/2025 10:27 AM	File folder	
cmake	5/7/2025 10:27 AM	File folder	
doc	5/7/2025 10:27 AM	File folder	
doc_en	5/7/2025 10:27 AM	File folder	
include	5/7/2025 10:27 AM	File folder	
N706B.si4project	5/7/2025 4:53 PM	File folder	
nwy_test_demo	5/7/2025 10:27 AM	File folder	
out	5/8/2025 4:30 PM	File folder	
prebuilts	5/7/2025 10:27 AM	File folder	
tools	5/7/2025 10:28 AM	File folder	
build_open_app_cmake.bat	5/7/2025 10:27 AM	Windows Batch File	3 KB
CMakeLists.txt	5/7/2025 10:27 AM	Text Source File	6 KB
cmd	5/7/2025 10:27 AM	Shortcut	2 KB
readme.txt	5/7/2025 10:27 AM	Text Source File	2 KB

As a developer, from the perspective of compilation and development, the most important things to focus on are the build_open_app_cmake.bat compilation script, the nwy_test_demo example file, and the [CMakeLists.txt](#) source link rule file under the SDK root directory.

File name	Description
build	Static library .a files and object .o files generated during compilation
include	Header files required for the sample code
nwy_test_demo	Sample code
/nwy_test_demo/ nwy_function_demo	Example code, TCP, network, virtual AT, and other functions
/nwy_test_demo/ nwy_test_cli	Example code, including all features
/nwy_test_demo/.gitignore	Git files, customers do not need to worry about them
CMakeLists.txt	Compile rule file
release	Files generated by the SDK compilation, including the APP, map

	mapping files, and app FOTA upgrade package.
tools	Some additional scripts and tools used for compilation
build_open_app_cmake .bat	The SDK upper-layer APP compilation script usually does not need to be modified.
cmd	cmd command line shortcuts

4 Application Example

You can develop software based on the SDK APIs provided by Neoway. Please refer to the API documentation for details.

4.1 Code Entry

In the /build directory of the SDK, the script in nwy_open_app_gnu.mak specifies the folder to be compiled, so /nwy_test_demo/nwy_test_cli is the C file that needs to be compiled for this project. The nwy_open_app_entry is defined in nwy_test_cli_adpt.c, which serves as the entry point for the entire code. There can only be one nwy_open_app_entry entry function in a compilation project.

```
1 TARGET_ = nwy_open_app
2
3 CC      = arm-none-eabi-gcc.exe
4 LINK    = arm-none-eabi-ld.exe
5 FROMELF = arm-none-eabi-objcopy.exe
6
7 lds_dir = build
8
9 obj_dir = obj
10
11 lds_file = nwy_open_app.lds
12
13 TARGET = ${TARGET_}
14
15 ifndef proj_dir
16     proj_dir = nwy_test_demo/nwy_test_cli
17 else
18     proj_dir = nwy_test_demo/nwy_function_demo
19 endif
20
21 include nwy_test_demo/Makefile.mk
22 VPATH = $(addprefix $(test_dir)/, $(dir $(srcs)))
23
24 VPATH = $(addprefix $(proj_dir)/, $(dir $(srcs)))
25 inc_dir_full +=--Iinclude
26 inc_dir_full +=--Iinclude/mbdctl
27 inc_dir_full +=--Iinclude/mbdctl/include
```

The specific implementation of nwy_open_app_entry is as follows:

```

int nwy_open_app_entry()
#else
int appimg_enter(void *param)
#endif
{
    nwy_thread_sleep(5 * 1000);
    char APP_BUILD_TIME[65] = {0};
    char version[70] = {0};

    sprintf(version, "%s", APP_VERSION);
    sprintf(APP_BUILD_TIME, "%s", __DATE__ __TIME__);
    NWY_CLI_LOG("nwy_open_app_enter ...");

    nwy_thread_create(&nwy_test_cli_thread, "test-cli", NWY_OSI_PRIORITY_NORMAL, nwy_test_cli_main_func, NULL, 16, 1024 * 32, NULL);
    return 0;
}

```

4.2 Task Creation

As outlined earlier, the ThreadX system is task-driven, and all user development is based on the creation and scheduling of tasks.

In the provided example, a task named 'test-cli' is created within the `nwy_open_app_entry` entry function. This task specifies the corresponding entry function, priority, and stack size. The entry function for this task, 'nwy_test_cli_main_func', encompasses almost all demo programs that can be single-stepped for debugging.

```

nwy_thread_create(&nwy_test_cli_thread, "test-cli", NWY_OSI_PRIORITY_NORMAL,
nwy_test_cli_main_func, NULL, 16, 1024 * 32, NULL);

```

```

1:
2: static void nwy_test_cli_main_func(void *param)
3: {
4:     char *sptr = NULL;
5:     nwy_usb_serial_reg_rcv_cb((nwy_sio_rcv_cb_t)nwy_test_cli_sio_data_proc);
6:
7:     while (1)
8:     {
9:         nwy_test_cli_menu_display();
10:        sptr = nwy_test_cli_input_gets("\r\nPlease input option:");
11:        if (sptr[0] == 'q' || sptr[0] == 'Q')
12:        {
13:            nwy_test_cli_menu_back();
14:        }
15:        else
16:        {
17:            nwy_test_cli_menu_select(atoi(sptr));
18:        }
19:    }
20: }
21:
22: ...

```

4.3 Serial Port Application

4.3.1 Serial Port demo Instructions

The functions for the serial port section of the N706 module can be designed with reference to the UART section in the main task. The N706 module can provide up to 2 serial ports for users. They are

the MAIN serial port and AUX serial port on the hardware guide. If the model includes GNSS, the AUX serial port cannot be used.



The correspondence between the serial port number initialized in the SDK and the pins of the N706 module is as follows:

- The NWY_NAME_UART2 variable corresponds to PIN 17 and PIN 18 pins, which are labeled as the UART1 serial port on the EVB.
- The NWY_NAME_UART3 variable corresponds to PIN 28 and PIN 29 pins, which are labeled as the UART2 serial port on the EVB.

DEMO nwy_test_cli_uart_init is a serial port initialization function setting. You can refer to this function to set the serial port type.

```
void nwy_test_cli_uart_open()
{
    char *opt;
    char* namelist[3] = {"URT1", "URT2", "URT3"};
    uint8_t port;
    uint32_t baudrate;
    uint8_t flowctrl;
#ifdef NWY_EC618_UART_OPEN_TEST
    opt = nwy_test_cli_input_gets("\r\nPlease input the uart id(1-URT0, 2-URT1 or 3-URT2):");
#else
    opt = nwy_test_cli_input_gets("\r\nPlease input the uart id(1-URT1, 2-URT2 or 3-URT3):");
#endif
    port = atoi(opt);
    if(port < 1 || port > 3)
    {
        nwy_test_cli_echo("\r\n Input UART id is invalid!");
        return;
    }
    name = namelist[port-1];
    nwy_test_cli_echo("\r\nTest port = %d!\r\n", port); //delet

    opt = nwy_test_cli_input_gets("\r\nPlease input the uart baudrate (4800, 9600, 19200,
38400, 57600, 115200, 230400, 460800, 921600):");
    baudrate = atoi(opt); // Set the baud rate
    nwy_test_cli_echo("\r\n Input UART baudrate = %d!\r\n", baudrate);

    opt = nwy_test_cli_input_gets("\r\nPlease input the uart flowctrl(1-FC_RTSCTS, 2-
FC_XONXOFF):"); //Set flow control
    flowctrl = atoi(opt);
    nwy_test_cli_echo("\r\n Input UART flowctrl = %d!\r\n", flowctrl);

    hd = nwy_uart_open(name, baudrate, flowctrl); // Serial port initialization
    if (hd < 0)
        nwy_test_cli_echo("\r\nTest uart error!\r\n"); //delet
    nwy_test_cli_echo("\r\nTest uart success!\r\n"); //delet
}
```

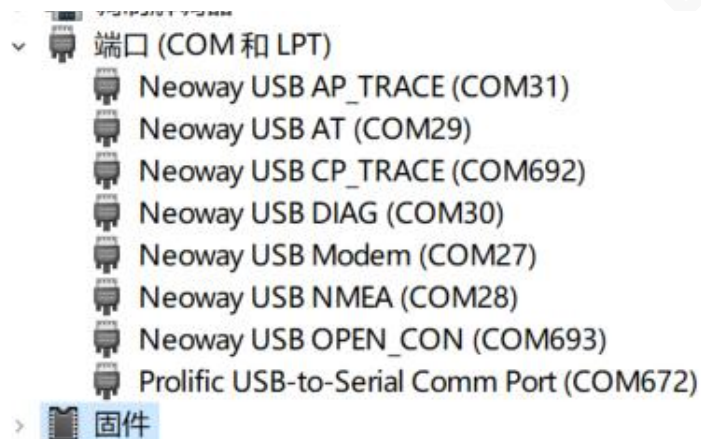
When writing serial port code, after the serial port initialization returns successfully, the `nwy_uart_open` interface needs to be used to set the initialized serial port ID, baud rate, and flow control.

If the serial port is set to data mode, then when a data reception triggers the corresponding serial port interrupt, it can be handled within the handle function. The maximum buffer size for serial port data transmission and reception is 4K bytes. The timeout for sending and receiving is 32ms. If you need to modify the reception timeout, you can use the `nwy_uart_rx_frame_timeout_set` interface to make changes.

4.3.2 Serial Port Test Example

The source code in `nwy_test_cli_uart.c` provides a reference demo for `nwy_test_cli_uart_open` and `nwy_test_cli_uart_write` functions. When you test with the development board, you can open two serial port tools to simulate data transmission and reception between the module's serial port and the PC's serial port. The following uses PIN17 and PIN8 pins of UART1 as an example.

Step 1: Refer to the EVK user guide. Use the USB cable and power serial board to connect the EVB to the PC. The power serial board should be connected to the UART1 interface. After the module is powered on, the relevant ports will be displayed in the device manager as shown below.



Step 2: Open the serial port tool, select the USB OPEN port (COM693), and enter the main function of `nwy_test_cli_main_func`. In the 3. Driver & Kernel → 1. Peripherals → 1. UART → 1. Uart open option, perform serial port initialization.

```
Please input option:
[17:05:51.533]OUT->1
□
[17:05:51.539]IN->◆
1
1. Uart open
2. Uart read
3. Uart write
4. Uart set para
5. Uart get para
6. Uart close
7. Uart rx callback register
8. Uart tx callback register
9. Uart rx frame timeout set
```



```

Please input option:
[17:06:17.914]OUT->1
□
[17:06:17.920]IN->◆
1
Please input the UART ID (1-URT2, 2-URT3): // Initialize the UART ID. The UART1 on the
EVB corresponds to NWY_NAME_UART2 in the OPEN code.
[17:06:19.609]OUT->1
□
[17:06:19.615]IN->◆
1
Test port = 1!

Please input the UART baud rate (4800, 9600, 19200, 38400, 57600, 115200, 230400, 460800,
921600): // Set baud rate
[17:06:24.406]OUT->115200
□
[17:06:24.412]IN->◆
115200
Input UART baudrate = 115200!

Please input the UART flow control (0-FC_NONE, 1-FC_RTSCCTS, 2-FC_XONXOFF): // Set UART
flow control
[17:06:27.080]OUT->0
□
[17:06:27.087]IN->◆
0
Input UART flowctrl = 0!

Test UART success! // Serial port initialization successful

1. Uart open
2. Uart read
3. Uart write
4. Uart set para
5. Uart get para
6. Uart close
7. Uart rx callback register
8. Uart tx callback register
9. Uart rx frame timeout set

[17:07:08.592]OUT->7 //Register serial port receive callback
□
[17:07:08.599]IN->◆
7
1. Uart open
2. Uart read
3. Uart write
4. Uart set para
5. Uart get para
6. Uart close
7. Uart rx callback register
8. Uart tx callback register
9. Uart rx frame timeout set

Please input option:
[17:07:12.273]OUT->8 //Register serial port receive callback
□
[17:07:12.279]IN->◆
8
Uart send complet handle success!

```

```
1. Uart open
2. Uart read
3. Uart write
4. Uart set para
5. Uart get para
6. Uart close
7. Uart rx callback register
8. Uart tx callback register
9. Uart rx frame timeout set
```

Please input option:

Step 3: Open another serial port tool and select the port corresponding to the connection between the module UART and the PC (e.g., COM672).

The following is the printout from the external serial port tool:

```
[17:11:08.394]OUT->◇1234567890 //Sending data to the module UART via serial port tool
□
[17:11:08.401]IN->◆1234567890 // After the module serial port receives the data, it sends
the same data to the PC serial port
```

The following is the printout from the OPEN port of the module:

```
[17:11:08.408]IN->◆
Uart send complete handle success! //Module side UART data sent successfully

Uart send data length = 12 //Received external serial port data length
```



The port number configured in `nwy_uart_open` during serial port initialization needs to be configured with reference to the pin correspondence in the 4.3.1 'Serial Port Demo Instructions' section.

4.4 Timer

The Timer-related API definitions can be found in `nwy_osi_api.h`.

```
//timer API
// Timer initialization, every timer needs to be initialized
nwy_error_e nwy_sdk_timer_create(nwy_osi_timer_t *timer, nwy_timer_para_t *para);
nwy_error_e nwy_sdk_timer_destroy(nwy_osi_timer_t timer); // Release timer resources
nwy_error_e nwy_sdk_timer_start(nwy_osi_timer_t timer, nwy_timer_para_t *para); // Start the
timer
nwy_error_e nwy_sdk_timer_stop(nwy_osi_timer_t timer); //Stop the timer
```

The activation of any timer requires initialization first, including defining the task name to be executed and the corresponding callback function. For example, during the creation of the main task, the timer is initialized, and a callback function is registered to be called when the timer times out.

```
ret = nwy_sdk_timer_start(s_nwy_test_timer, &para);  
if (ret != NWY_SUCCESS) {  
    nwy_test_cli_echo("\r\nstart timer fail");  
} else {  
    nwy_test_cli_echo("\r\nstart timer succes");  
}
```

Currently, there is no limit on the number of Timers that can be created at the upper level. However, it is still recommended to promptly deinit any Timers that have been requested but are not in use to avoid occupying excessive system resources.

4.5 FLASH Address Related

The N706B OpenCPU module is equipped with 4MB ROM + 4MB RAM, and the defined FLASH space is as follows. If you have special customization needs, contact FAE for assistance.

In the SDK's \tools\aboot\configurations\releasepack-asr1605-source\config\partition path, you can see a file named ASR1605_SMS_SINGLE_FLASH_LAYOUT.json. This file contains information about the remaining space in the file system and the maximum compilation space for the app corresponding to the SDK. This JSON file is read-only and cannot be modified; otherwise, it will cause other errors.

In the ASR1605_SMS_SINGLE_FLASH_LAYOUT.json file, the keyword 'nvm' represents the usable space of the file system, and 'app' represents the size of the compiled app. Taking the N706B-A06-STD-OE_CN1X-001 version corresponding SDK as an example, the available space in the file system is 152KB, and the compilable app size is 256KB.

```
"partitions": [  
  {  
    "name": "nvm",  
    "format": "raw",  
    "size": "152KiB"  
  },  
  {  
    "name": "fota_param",  
    "format": "raw",  
    "size": "12KiB"  
  },  
  {  
    "name": "fota_pkg",  
    "format": "raw",  
    "size": "4KiB",  
    "disable": true  
  }  
],  
{  
  "name": "app",  
  "format": "raw",  
  "size": "256KiB"  
},  
]
```

5 Debugging

To facilitate customer debugging, Neoway provides a LOG printing interface, which can direct debugging prints to the USB port through the provided APIs.

The log printing interface is defined by `nwy_usb_serial.h`. Use the `nwy_test_cli_echo()` function to print relevant information to the Neoway USB OPEN_CON's USB COM port.

```
void nwy_test_cli_echo(char *fmt, ...)
{
    static char echo_str[NWY_CLI_ECHO_LEN];
    static nwy_osi_mutex_t *echo_mutex = NULL;
    va_list a;
    int i, size;

    if (NULL == echo_mutex) {
        nwy_sdk_mutex_create(&echo_mutex);
    }
    if (NULL == echo_mutex) {
        return;
    }
    nwy_sdk_mutex_lock(echo_mutex, NWY_OSA_SUSPEND);
    va_start(a, fmt);
    vsnprintf(echo_str, NWY_CLI_ECHO_LEN, fmt, a);
    va_end(a);
    size = strlen((char *)echo_str);
    i = 0;
    while (1)
    {
        int tx_size;

        tx_size = nwy_usb_serial_send((char *)echo_str + i, size - i);
        if (tx_size <= 0)
            break;
        i += tx_size;
        if ((i < size))
            nwy_thread_sleep(10);
        else
            break;
    }

    nwy_sdk_mutex_unlock(echo_mutex);
}
```